

Domain-Driven Design in Software Development: A Systematic Literature Review on Implementation, Challenges, and Effectiveness

OZAN ÖZKAN, Mathematics and Computer Science, Eindhoven University of Technology, The Netherlands
ÖNDER BABUR, Information Technology Group, Wageningen University & Research, The Netherlands
and Mathematics and Computer Science, Eindhoven University of Technology, The Netherlands
MARK VAN DEN BRAND, Mathematics and Computer Science, Eindhoven University of Technology,
The Netherlands

Context: Domain-Driven Design (DDD) addresses software challenges, gaining attention for refactoring, reimplementation, and adoption. It centers on domain knowledge to solve complex business problems.

Objective: This Systematic Literature Review (SLR) analyzes DDD research in software development to assess its effectiveness in solving architecture problems, identify challenges, and explore outcomes.

Method: We selected 36 peer-reviewed studies and conducted quantitative and qualitative analysis.

Results: DDD effectively improved software systems, emphasizing Ubiquitous Language, Bounded Context, and Domain Events. DDD in microservices gained prominence for system decomposition. Some studies lacked empirical evaluations, identifying challenges in onboarding and expertise.

Conclusion: Adopting DDD benefits software development, involving stakeholders like engineers, architects, managers, and domain experts. More empirical evaluations and open discussions on challenges are needed. Collaboration between academia and industry advances DDD adoption and knowledge transfer in projects.

CCS Concepts: • **Software and its engineering** → **Software development techniques**.

Additional Key Words and Phrases: Domain-Driven Design (DDD), Software Architecture, Software Development, Systematic Literature Review

ACM Reference Format:

Ozan Özkan, Önder Babur, and Mark van den Brand. 0000. Domain-Driven Design in Software Development: A Systematic Literature Review on Implementation, Challenges, and Effectiveness. 1, 1 (November 0000), 29 pages. <https://doi.org/XXXXXXX.XXXXXXX>

1 INTRODUCTION

In the field of software development, the pursuit of effective design methodologies and architectural approaches has been a crucial endeavor for both researchers and practitioners. Among these approaches, Domain-Driven Design (DDD) has gained significant attention and adoption, particularly in the context of solving software architecture problems and addressing challenges related to refactoring, reimplementation, and adoption [2]. DDD, as advocated by Eric Evans in his influential book, *"Domain-Driven Design: Tackling Complexity in the Heart of Software"* [13], offers

Authors' addresses: Ozan Özkan, Mathematics and Computer Science, Eindhoven University of Technology, Eindhoven, The Netherlands, o.ozkan@tue.nl; Önder Babur, Information Technology Group, Wageningen University & Research, Wageningen, The Netherlands and Mathematics and Computer Science, Eindhoven University of Technology, Eindhoven, The Netherlands, o.babur@tue.nl, onder.babur@wur.nl; Mark van den Brand, Mathematics and Computer Science, Eindhoven University of Technology, Eindhoven, The Netherlands, m.g.j.v.d.brand@tue.nl.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 0000 Association for Computing Machinery.

XXXX-XXXX/0000/11-ART \$15.00

<https://doi.org/XXXXXXX.XXXXXXX>

a comprehensive framework for understanding, modelling, and solving complex business problems by placing domain knowledge at the core of the design process.

DDD emphasizes the alignment of software systems with the underlying business domain, enabling developers to better understand, model, and address software architecture challenges effectively [13, 49]. By capturing the language, concepts, and relationships within the domain, DDD enables the identification of architectural problems and provides guidance on how to refactor, reimplement, and adopt DDD principles to improve system quality, maintainability, and scalability.

While DDD has garnered praise for its conceptual elegance and potential benefits in addressing software architecture challenges, it is crucial to evaluate its practical application and assess its impact on software development projects. This Systematic Literature Review (SLR) aims to provide a comprehensive analysis of existing research studies that have utilized DDD for refactoring, reimplementation, adoption, and other related purposes in software development. By analyzing and synthesizing empirical studies, we aim to gain insights into the effectiveness of DDD in solving software architecture problems, identify the challenges encountered during the application of DDD, and explore the outcomes and lessons learned from these studies.

To achieve this objective, a rigorous research methodology adhering to established guidelines proposed by Kitchenham [22] and Wohlin [56] were employed. A pool of 36 peer-reviewed studies was systematically selected, and both quantitative and qualitative analyses were conducted based on the data extracted from these studies.

The conclusion drawn from this SLR emphasizes the potential benefits of adopting DDD in software development. The study highlights the importance of involving key stakeholders, including software engineers, architects, project managers, and domain experts, for successful implementation. Moreover, the study identifies the importance of open discussions about challenges and limitations and emphasizes collaboration between academia and industry for advancing DDD adoption and knowledge transfer in real-world projects.

This study contributes to the growing body of knowledge on DDD and provides valuable insights for practitioners and researchers seeking to apply DDD principles effectively in software development. By identifying the strengths and weaknesses of existing research and proposing areas for improvement, this SLR aims to foster the further advancement and adoption of DDD principles in contemporary software development projects.

1.1 Domain-Driven Design (DDD) Essentials

Domain-Driven Design (DDD) is a software development methodology and conceptual framework in software engineering that seeks harmony between the structure of software code and the complexities of the business domain it serves. The goal of DDD is to bridge the gap between technical implementation and business requirements, resulting in more effective communication between developers and domain experts. This concept was pioneered by Eric Evans, whose significant book *Domain-Driven Design: Tackling Complexity in the Heart of Software* [13] has shaped this paradigm. Evans emphasizes the alignment of software design with the intricate nuances of the business domain. He argues that effective software development requires a deep understanding of the domain's intricacies, enabling developers to create software that genuinely resonates with the real-world needs [13].

In DDD, the domain refers to the specific area of expertise or the subject matter of the software application. This could be anything from e-commerce, finance, healthcare, or logistics. The central concept in DDD is the development of a rich, well-defined domain model that accurately represents the business rules, processes, and entities within the domain. This model is built through collaboration between developers and domain experts, often using a shared language that captures both technical and domain-specific terms.

Through its concepts, which are explained in the following sections, DDD orchestrates a symphony of principles that empower software architects to sculpt solutions that mirror the multifaceted dimensions of the real world. By embracing these core tenets, software engineers navigate the complex landscape of software development with a compass calibrated to the nuances of the domain, thus achieving a symbiotic fusion of technical precision and domain understanding.

1.1.1 Ubiquitous Language. At the core of DDD lies the concept of Ubiquitous Language, an ingenious mechanism to bridge the gap between technical jargon and domain-specific vocabulary [13]. Ubiquitous Language fosters a shared vocabulary that harmonizes conversations among developers, domain experts, and business stakeholders. By encapsulating the complexities of the business domain in a common understanding, Ubiquitous Language eliminates ambiguity and streamlines communication. This shared linguistic framework empowers stakeholders to articulate domain intricacies effortlessly and translates directly into the software's code, facilitating a direct and unambiguous translation of domain understanding into the software's design and implementation.

1.1.2 Bounded Context. The notion of Bounded Context, manifests as a cornerstone of DDD. Bounded Context delineates discrete zones within which the semantics of the domain model remain consistent. In a large-scale application encompassing diverse domain models, the risk of confusion and misalignment arises. Bounded Contexts act as conceptual partitions, segregating distinct domain models to preserve integrity. By confining the domain to specific contexts, this concept mitigates the potential for semantic clashes and maintains a cohesive representation of the business domain within each bounded region [13].

1.1.3 Aggregates. The concept of Aggregates is a pivotal construct in DDD, which encapsulate entities, value objects, and domain logic. Aggregates serve as focal points for data access and changes, ensuring data integrity within business transactions [13]. By containing data within defined boundaries, Aggregates optimize data management and system scalability, while also enforcing logical boundaries that prevent uncontrolled data manipulation.

1.1.4 Entities and Value Objects. Entities and Value Objects differentiates within the DDD framework. Entities embody tangible elements within the domain, each possessing a unique identity. These entities undergo changes over time, capturing the mutable aspects of the business reality [13]. In contrast, Value Objects represent conceptual attributes that lack identity and remain immutable. By distinguishing between these two constructs, DDD affords a comprehensive representation of the domain, accurately modeling both tangible entities and abstract attributes [13].

1.1.5 Domain Services. Domain Services encapsulate domain logic that transcends individual entities and value objects [13]. These services orchestrate complex operations and interactions within the domain. By segregating multifaceted logic, domain services promote modular, reusable, and concise code. This approach mirrors the coherent coordination of real-world processes, enhancing the software's alignment with the domain's intricacies.

1.1.6 Domain Events. In DDD, Domain Events are a crucial concept used to model and communicate changes or significant occurrences within a domain [49]. They play an important role in capturing and representing the behavior and state changes that happen within the core domain of an application. A Domain Event is a lightweight object that represents something important that has happened in the domain. These events are usually named in the past tense to indicate that they've already occurred. They help decouple different parts of a system by allowing different components or aggregates to communicate without needing to know the specifics of each other. This promotes a more modular and maintainable architecture.

1.1.7 Anti-Corruption Layer. The concept of an Anti-Corruption Layer, safeguards the domain model from external influences. This layer mediates interactions between different models or systems, preserving the domain's integrity. By translating data between disparate representations, the Anti-Corruption Layer ensures a consistent domain model, safeguarding it from external distortions [13].

The DDD methodology's fundamental contribution lies in the creation of software systems that closely mirror the cognitive schema of the business. By converging domain experts, business stakeholders, and software developers, DDD facilitates a synergistic collaboration aimed at delivering software solutions that optimally cater to business needs. This collaborative effort fosters the combination of domain insights, augments the comprehension of problems and solutions across the entirety of the development team, and supports fostering of inter-team relationships.

2 RESEARCH OBJECTIVES AND METHOD

This study employed a Systematic Literature Review (SLR) methodology to consolidate the existing knowledge pertaining to the utilization of DDD in the domain of software development, aiming to comprehensively assess the current status of research in this field. The research approach adhered to established guidelines as proposed by Kitchenham [33] and Wohlin [56].

Additionally, this study delves into further detail regarding the Goal and Research Questions (Section 2.1), Search Strategy and Data Sources (Section 2.2), Exclusion Criteria (Section 2.3), Primary Study Identification and Selection (Section 2.4), and Data Extraction (Section 2.5) in subsequent sections. By examining these aspects in a meticulous manner, this study aims to provide a comprehensive and rigorous analysis of the research topic at hand.

2.1 Goal and Research Questions

The primary objective of this research is to investigate and synthesize the current state of research on the application of DDD in software development, to understand its benefits, challenges, techniques, and effectiveness. To achieve this objective, the following research questions (RQ)s have been formulated:

- RQ1:** What is the current state of research concerning DDD in the existing literature?
- RQ2:** What categories of software systems have demonstrated benefits from the implementation of DDD?
- RQ3:** Which specific software development problems have been addressed through the application of DDD in previous studies?
- RQ4:** What are the most common DDD techniques utilized in previous research, such as Anti-Corruption Layers, Bounded Contexts, and other related approaches?
- RQ5:** What are the challenges encountered during the implementation of DDD in software development projects?
- RQ6:** How has the effectiveness of DDD been assessed and measured in prior studies?
- RQ7:** Who are the key stakeholders involved and benefited in the implementation of DDD?

2.2 Search Strategy and Data Sources

In line with established scholarly protocols, we meticulously designed our search strategy to identify pertinent studies from reputable academic databases renowned for their extensive coverage of computer science and software engineering literature. The databases utilized in this study were *ACM*, *SpringerLink*, *IEEE*, and *Wiley*, all of which are widely recognized for their authoritative and diverse repositories of scholarly articles.

Table 1. List of Exclusion criteria

#	Exclusion Criteria
EC1	Duplicate papers from multiple sources.
EC2	Papers without full text available.
EC3	Papers not written in English.
EC4	Books, book chapters, editorials, issue introductions.
EC5	Papers which are not a primary study.
EC6	Papers don't explicitly encapsulate introducing a method/technique/tool/evaluation of adaptation/extension.

The search query employed in this investigation was thoughtfully crafted to encompass a broad spectrum of articles relevant to our research objectives. The query utilized a combination of keywords and logical operators to target studies explicitly discussing Domain-Driven Design in conjunction with activities related to refactoring, improvement, optimization, restructuring, migration, evolution, cleanup, and reengineering. The search query was formulated as follows:

"Domain Driven Design" AND (refactor* OR improv* OR "optimiz*" OR "restructur*" OR "migrat*" OR "evolution" OR "cleanup" OR "reengineering")

We strategically used asterisks (*) as wildcards to capture variations of the specified keywords and to ensure inclusivity in the search results.

2.3 Exclusion Criteria

The establishment of selection criteria was undertaken with the aim of mitigating bias and minimizing subjectivity within the study. The inclusion and exclusion criteria were formulated in accordance with the research questions (RQs) as delineated by Kuhrmann et al. [26], thereby ensuring that the criteria aligns closely with the intended objectives of the investigation. Following our criteria, (1) we meticulously identified and removed duplicate studies that appeared in more than one of the selected databases to prevent the inclusion of redundant information. To maintain the comprehensiveness of our analysis, (2) we excluded studies that lacked full-text availability, as the absence of complete content may hinder a thorough evaluation of the study's findings and methodology. Given the language limitations of our review team, (3) we restricted our analysis to studies written in English to ensure accurate and consistent interpretation and analysis. In the interest of focusing on primary research articles, (4) we excluded books, book chapters, editorials, and issue introductions from our analysis, as they may not present original empirical studies or detailed investigations. (5) We excluded studies that were not primary research studies (e.g., review articles, meta-analyses, opinion pieces) to maintain a clear focus on empirical research directly relevant to our research questions. To ensure the inclusion of studies closely related to DDD and its practical applications, (6) we excluded studies that did not explicitly present or discuss the introduction of a method, technique, tool, or evaluation pertaining to DDD or its adaptation and extension. The list of the exclusion criteria can be seen in Table 1.

2.4 Primary Study Identification and Selection

2.4.1 *Conducting Manual Database Search.* In the first stage, we performed a comprehensive manual search of the databases mentioned in Section 2.2. The search was conducted in March 2023 and yielded a total of 707 studies, which were 153 studies from ACM, 19 studies from IEEE Xplore, 510 studies from SpringerLink, and 25 studies from Wiley, as also shown on Table 3. To ensure the

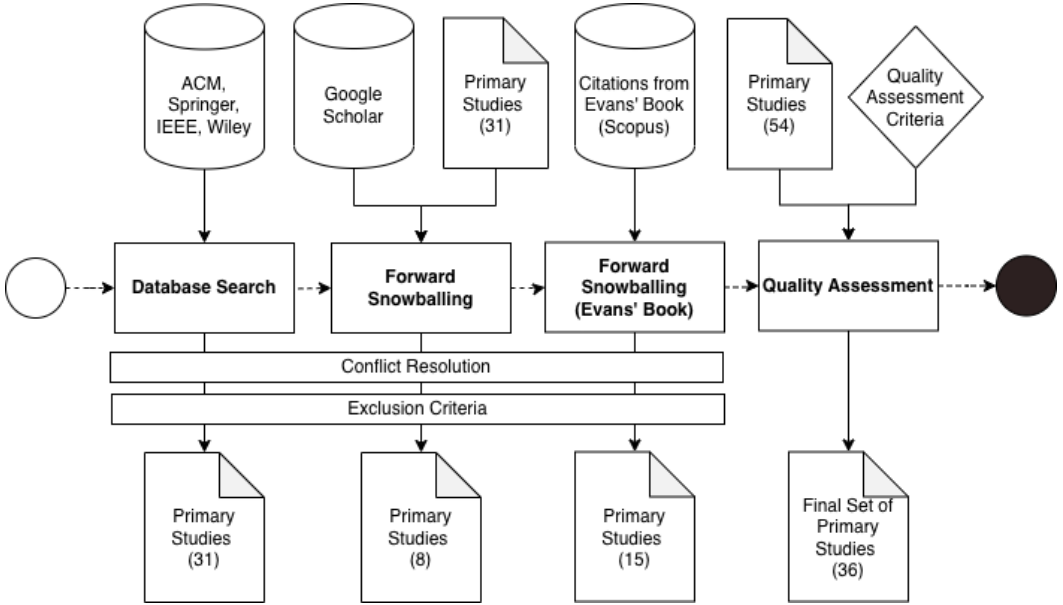


Fig. 1. Study selection progress from a database search to the quality assessment. The final set of primary studies advanced to the data extraction stage.

exclusion of duplicate results, we conducted conflict resolution, resulting in the identification of 1 duplicate paper. After eliminating duplicates, we advanced 706 unique studies to the next stage of the selection process.

2.4.2 Applying Exclusion Criteria. In the second stage, we applied our pre-defined exclusion criteria to the 706 studies. The screening process involved the assessment of each paper based on its title, abstract, and keywords. In situations where a thorough evaluation was not possible based on these elements, we briefly reviewed the introduction and conclusion of the paper, following the guidelines suggested by Wohlin [56]. To ensure a rigorous assessment, we adopted a multi-assessor strategy, where two authors independently evaluated the studies without knowing each other's assessments. Due to the high number of studies, we performed the assessment in iterations until we achieved at least 0.80 Cohen's Kappa coefficient [8], which is indicating almost perfect agreement between the assessors [8]. Once the desired Cohen's Kappa was reached, the first author continued to assess the remaining studies to determine their inclusion or exclusion, following the approach done in previous studies [47, 54]. In each iteration, we randomly selected 30 studies for assessment. After each iteration, we conducted conflict resolution sessions to discuss our assessments' rationale. We eventually achieved 1.0 Cohen's Kappa coefficient in the second iteration. The remaining studies were assessed by the first author resulting in 31 studies marked for inclusion and advancement to the next stage. The distribution of these included studies across the search databases is presented in Table 3.

2.4.3 Conducting Forward Snowballing. In the third stage, we conducted forward snowballing on the included studies. Following Wohlin's guidelines [56], we used Google Scholar to identify studies that cited our included primary studies. We assessed these additional studies based on their title, abstract, and keywords, using the exclusion criteria established earlier. In cases where a definitive resolution was not achievable based on the provided information, we conducted a rapid analysis of

Table 2. List of quality criteria adopted from Kitchenham’s guidelines [22]. Each criteria is rated on a scale of 0 to 1. Studies that achieved a score of ≥ 4.5 advanced to the data extraction stage.

#	Quality Criteria
QC1	The aim of the study is clearly stated.
QC2	Scope and Context clearly defined.
QC3	Variable, valid and reliable.
QC4	The research process is documented properly.
QC5	The research questions are answered.
QC6	Negative findings presented.
QC7	The main findings are clearly stated.
QC8	Conclusions relate to the aim of the purpose of the study.

the entire paper, as suggested by Wohlin [56]. Through this process, we identified 8 studies that met the inclusion criteria and were added to our study.

Additionally, to make our study population wider and ensure relevancy, we conducted a forward snowballing process for studies that cited Evans’ book [13]. Eric Evans holds significant prominence in the field of DDD due to his influential contributions. His book, which serves as a seminal reference, has gained substantial recognition and is widely regarded as a crucial resource among practitioners in the software development community [2, 36, 49, 50]. To conduct this snowballing, we utilized Scopus as a database because of its centralized indexing of our target databases mentioned in Section 2.2 and to eliminate results from grey literature. The screening process for Evans’ book forward snowballing followed the same strategy as the primary studies’ forward snowballing. In the process of analysis, a comparison was made among the database search results, forward snowballing results, and the previously identified studies to identify any potential duplicates. As a result, a total of 10 duplicate studies were identified, and these duplicates were subsequently resolved in favour of the database search results and forward snowballing outcomes. Consequently, an additional 15 studies that fulfilled the inclusion criteria were identified and subsequently included in our study. The comprehensive forward snowballing process, encompassing both the forward snowballing of included studies and the examination of studies cited in Evans’ book, resulted in a total of 23 studies being incorporated into our analysis.

2.4.4 Quality Assessment. During stage four of our primary study identification and selection process, we proceeded with the task of conducting a comprehensive quality assessment of the selected studies. Adhering to the established guidelines put forth by Kitchenham et al. [22], each study was meticulously evaluated, with ratings assigned on a scale ranging from 0 to 1. On this scale, a rating of 1 denoted a positive affirmation, 0.5 indicated a partial fulfilment of the quality criteria, and 0 represented a negative response. The specific quality assessment questions utilized in this evaluation can be found in Table 2. Studies that achieved a total score of 4 or higher were deemed to have successfully met the quality assessment criteria and were subsequently included in the data extraction stage. In order to maintain the rigour and objectivity of the process, we also adopted a multi-assessor strategy during this stage. Similar to the exclusion criteria stage, two independent researchers were tasked with individually evaluating the quality of each study, and at no point were they made aware of each other’s assessments. This approach was designed to minimize potential biases and enhance the reliability of the quality assessment outcomes.

Table 3. Primary Study Identification Metrics

Database	Search Date	Results	Conflict	Included	Passed Quality
ACM	March 2023	153	0	13	12
IEEE Xplore	March 2023	19	0	8	3
SpringerLink	March 2023	510	1	8	5
Wiley	March 2023	25	0	2	1
Totals Database Search		707	1	31	21
Scopus (Snowballing)	April 2023	608	10	15	9
Google Scholar (Snowballing)	June 2023	93	0	8	6
Totals Snowballing		701	10	23	15
Total		1408	11	54	36

2.5 Data Extraction and Synthesis

2.5.1 *Data Extraction.* In order to ensure consistency and alignment with our research questions and goals, a data extraction form was developed (Table 4). The initial list of data points was prepared by the first author, and this list was then discussed among the authors to refine and finalize a starting set of categories. A pilot data extraction was done on a subset of randomly selected primary studies. During the data extraction process, we adopted an iterative and incremental approach, continuously refining the categories as new insights emerged. This iterative refinement allowed us to adapt the data extraction form to better capture the nuances and specific details relevant to our RQs and goals.

The data points were categorized into three main categories. Firstly, the *Publication Metadata* category involved extracting basic information such as the title, authors, and publisher of each study. Secondly, the *Study Characteristics* category included data related to the study design, data collection methods, and evaluation metrics employed. Lastly, the *Domain-Driven Design* category focused on extracting information specifically pertaining to the application of DDD in each study.

Following the completion of the quality assessment stage, the final set of studies deemed relevant and has sufficient quality to be moved to the data extraction stage. The data extraction process was carried out by the first author using a predefined data extraction form. This form served as a structured tool to systematically collect and record pertinent information from the selected studies.

2.5.2 *Synthesis.* In this study, the data extraction process involved the collection of both qualitative and quantitative data from the final set of primary studies to derive our results. Specifically, for research questions (RQs) encompassing 1 to 4, we focused on extracting quantitative data. The frequencies and percentages of each identified category were then computed and reported to address these specific research questions. This approach allowed us to quantify and present the distribution and prevalence of different factors and concepts relevant to the research questions under consideration.

Conversely, for RQs pertaining to RQs 5 to 7, we concentrated on extracting qualitative data from the primary studies. To synthesize and interpret these qualitative findings, we applied a narrative synthesis method based on the guidance provided by Popay et al. [43]. This involved systematically extracting key information and important findings from each individual study. Subsequently, we conducted a comprehensive analysis and interpretation of the extracted qualitative data to draw overarching and meaningful conclusions in response to the respective research questions. The

Table 4. Data Extraction Form

Main Category	Field	Input Type	Relevant RQ
<i>Publication Metadata</i>	ID	Auto Alphanumeric	-
	Title	Free Text	RQ1
	Author	Free Text	RQ1
	Item Type	Selection	RQ1
	Database	Selection	RQ1
	Publication Year	Number	RQ1
	Publisher	Free Text	RQ1
	Source	Selection	RQ1
<i>Study Characteristics</i>	Study Design	Selection	RQ1
	Data Collection Methods	Free Text	RQ1
	Academic/Industry	Binary	RQ1
	Participants	Free Text	RQ7
	Empirically Evaluated	Binary	RQ1
	Human Metrics	Binary	RQ1
	Evaluation Methods	Free Text	RQ6
	Evaluation Metrics	Free Text	RQ6
<i>Domain-Driven Design (Context)</i>	Evaluation Results	Free Text	RQ6
	Software System	Free Text	RQ2
	Problems/Motivations	Free Text	RQ3
	Evaluation of Effectiveness	Free Text	RQ6
	Challenges and Limitations	Free Text	RQ5
	Lessons Learned	Free Text	RQ4-5
<i>Domain-Driven Design (Concepts)</i>	Ubiquitous Language	Binary	RQ4
	Bounded Context	Binary	RQ4
	Anti-Corruption Layer	Binary	RQ4
	Aggregation	Binary	RQ4
	Domain Events	Binary	RQ4
	Domain Services	Binary	RQ4
	Entities & Value Objects	Binary	RQ4

narrative synthesis approach allowed us to integrate diverse qualitative evidence, explore patterns and themes across the studies, and provide a cohesive and comprehensive understanding of the research questions in question.

3 RELATED WORK

To the best of our knowledge, despite the growing interest in DDD as a software development approach, there is a scarcity of SLRs specifically focused on DDD. While the literature landscape on

DDD is not completely unexplored, previous studies have primarily touched upon certain aspects remotely related to DDD.

For instance, Le [29] proposes a set of concrete design patterns for Domain Modeling within the context of Object-Oriented Domain-Driven Design, providing valuable insights into the practical application of DDD in specific modeling scenarios. Singjai [44], on the other hand, examines the interrelation between microservice APIs and DDD, shedding light on how DDD principles can be applied in the context of microservices architecture. However, it is worth noting that some related systematic literature reviews have been conducted in adjacent domains. Goey [11], for instance, undertakes a Systematic Literature Review on Design-Driven Innovation, exploring design-driven approaches and methodologies in innovation processes, which, although related to design-driven practices, do not directly address the specific principles and techniques of DDD. Similarly, Bertoni [3] conducts a Systematic Literature Review on Data-Driven Design in Concept Development, focusing on the role of data-driven approaches in the early stages of concept development but not explicitly addressing the nuances of DDD.

Consequently, the absence of a comprehensive literature review dedicated exclusively to DDD highlights the need for a systematic synthesis of the existing knowledge in this domain.

4 RESULTS

This section presents a comprehensive analysis of the findings obtained from our SLR conducted on Domain-Driven Design (DDD). This section aims to address the research questions by providing a detailed exploration of the current state of research pertaining to DDD in the literature. Through a meticulous examination of various articles, studies, and sources, the section sheds light on the key insights and trends related to the implementation, effectiveness, and assessment of DDD in diverse software systems. By organizing the results around the research questions, this section offers valuable insights into the involvement of key stakeholders, the evaluation and measurement of DDD’s effectiveness, and the overarching research landscape concerning this domain-centric architectural approach.

4.1 (RQ1) What is the current state of research concerning DDD in the existing literature?

Based on our primary studies (36), the current state of research concerning Domain-Driven Design (DDD) in the existing literature showcases studies conducted between 2006 and 2023. Among these studies, 23 (64%) are *Conference Papers*, 12 (33%) are *Journal Articles*, and 1 (3%) study is from the *Preprint* database, which also can be seen in Figure 2.

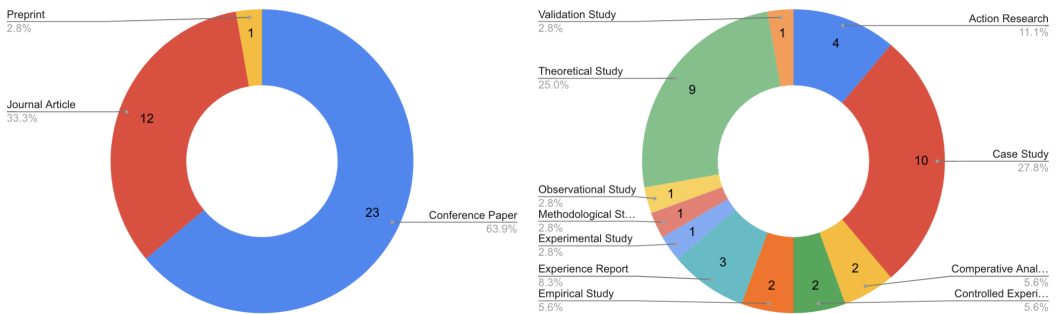


Fig. 2. Breakdown of study types (left) and study designs (right).

In terms of venues, a diverse range of conferences and journals have emerged as platforms for the dissemination of DDD research. These venues span the spectrum of software engineering and technology domains. Among them, the *Conference on Object-Oriented Programming Systems, Languages, and Applications* takes a prominent role with 3 publications, showcasing its relevance in fostering discussions around object-oriented paradigms in software development. Additionally, the *Symposium on Information and Communication Technology* have contributed 2 publications. The full list can be seen in Table 5.

The most common study type is *Case Study*, accounting for 10 (28%) studies, followed by *Theoretical Study* with 9 (25%) studies. There are also 4 (11%) *Action Research* studies, 3 (8%) *Experience Reports*, 2 (6%) *Empirical Studies*, 2 (6%) *Controlled Experiments*, 2 (6%) *Comparative Analyses*, 1 (3%) *Observational Study*, 1 (3%) *Methodological Study*, and 1 (3%) *Experimental Study*.

Regarding the software systems studied, *Microservices* received the highest attention with 16 (44%) studies, followed by *Enterprise Software Systems* with 6 (17%) studies, and DDD-related *Tools* and *System Agnostic* studies with 4 (11%) studies each. Additionally, *Standard Computer Applications* and *Web Applications* were subjects of 2 (6%) studies each, while *Service-Oriented Applications* and *Distributed Systems* each had 1 (3%) study.

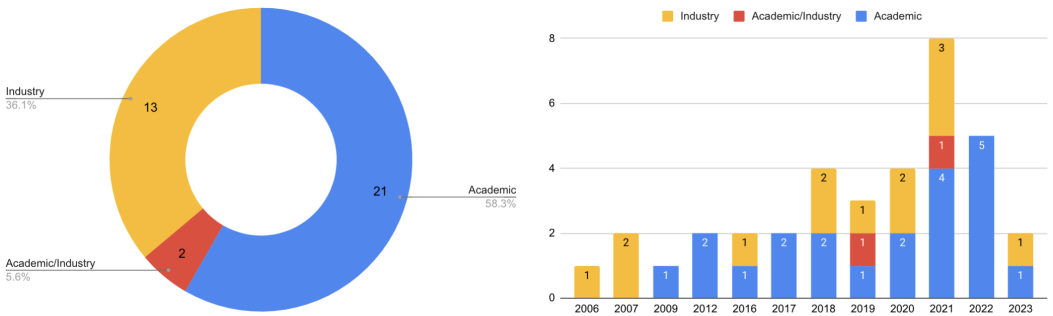


Fig. 3. Breakdown of the primary studies based on their field of focus (left) and field of focus by publication year (right).

Out of the 16 *Microservices* studies, 10 (63%) were conducted in an academic environment, while the remaining 6 (38%) were performed in industrial settings. In contrast, *Enterprise Software Systems* were predominantly studied in industry settings. Interestingly, DDD-related *Tools* were exclusively implemented in academic settings.

The interest in DDD initially emerged in industry settings, as evidenced by the first three studies conducted between 2006 and 2007. Subsequently, the focus shifted to academic settings, with all three studies from 2009 to 2012 being conducted in academia. The trend then became more balanced, with an equal distribution of studies between academic and industrial settings in 2016.

The distribution of studies across academic and industry settings reveals an interesting pattern. Out of the total 36 studies, 21 (58.3%) were conducted in academia, indicating a dominant presence of academic research in this field. Conversely, 13 (36.1%) studies were conducted in industry settings, and 2 (5.6%) studies had a hybrid setting involving both academic and industry collaboration. This disparity in the academic-industry ratio suggests that DDD has garnered significant attention from the academic community, where researchers are actively exploring its concepts and applications. However, it also highlights the relevance and practical implications of DDD in industry settings, as evidenced by the notable number of industry-based studies.

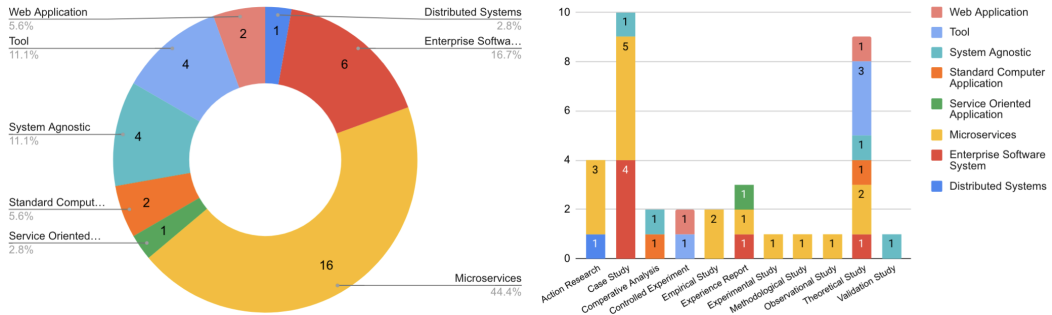


Fig. 4. Breakdown of the primary studies based on applied software systems (left) and studies grouped by the study type and software systems (right).

The existing literature demonstrates a diverse and evolving landscape of research on DDD, with a particular emphasis on Microservices and an increasing number of academic contributions. This suggests a growing recognition of the relevance and applicability of DDD in both academic and industrial contexts.

4.2 (RQ2) What categories of software systems have demonstrated benefits from the implementation of DDD?

With this RQ, we present the categories of software systems that have demonstrated benefits from the implementation of DDD, based on the findings. The studies were analyzed, and their contributions to various software system categories were identified during the data extraction phase. The breakdown of the following data can be also visually seen in Figure 4.

4.2.1 Microservices. Several studies have highlighted the advantages of applying DDD principles in the context of microservices architecture [1, 4, 7, 12, 16, 18, 19, 25, 35, 37, 39, 45, 52, 55, 57, 58]. According to our learnings from those studies, by adopting DDD, organizations have been able to improve the modularity, maintainability, and scalability of microservices-based applications, enabling effective handling of complex and distributed systems. The use of DDD patterns and concepts in the design and development of microservices has been particularly beneficial in achieving clear boundaries between services, enabling better communication between development teams, and ensuring a strong alignment with business domains.

4.2.2 Enterprise Software Systems. Several studies have reported the benefits of applying DDD in the context of enterprise software systems [10, 24, 27, 34, 38, 48]. The adoption of DDD principles has proven valuable in handling the increasing complexity of enterprise systems and addressing challenges related to legacy system modernization. By emphasizing bounded contexts and domain models, DDD enables more maintainable and flexible enterprise software architectures, allowing organizations to better adapt to changing business requirements.

4.2.3 Web Applications. Some studies have shown that DDD can be beneficial in the development of web applications [6, 32]. By applying DDD principles, web application developers have been able to improve the design and implementation of Single Page Applications (SPAs), achieving better cross-framework compatibility and enhancing productivity in SPA development.

4.2.4 Tools and Standard Computer Applications. Several studies have applied DDD in the context of tools [17, 20, 40] and standard computer applications [14, 31]. DDD has been leveraged to improve code quality, enhance the software construction process, and provide more comprehensive solutions

Table 5. List of venues and publication count per venue.

Venue	Publications
ACM SIGPLAN Conference on Object-Oriented Programming Systems, Languages, and Applications	3
International Symposium on Information and Communication Technology	2
Workshop on AOP and Meta-Data for Software Evolution	1
Symposium and Summer School on Service-Oriented Computing	1
Software	1
Journal on Information and Software Technology	1
Journal on Computer Languages, Systems & Structures	1
Journal of Software and Systems Modeling	1
International Symposium on Service Oriented Software Engineering	1
International Journal on Land Use Policy	1
International Journal on Empirical Software Engineering	1
International Journal on Advances in Software	1
International Conference on the Quality of Information and Communications Technology	1
International Conference on Software Engineering in Africa	1
International Conference on Model-Driven Engineering and Software Development	1
International Conference on Information Technology and Computer Application (ITCA)	1
International Conference on Information and Software Technologies	1
International Conference on Engineering and Emerging Technologies (ICEET)	1
International Conference on Dependable Systems and Their Applications (DSA)	1
International Conference on Agile Software Development	1
Informatik 2019	1
INCOSE International Symposium	1
IEEE International Conference on Software Architecture	1
IEEE Access	1
European Conference on Pattern Languages of Programs	1
European Conference on Computer Systems	1
Asia-Pacific Software Engineering Conference (APSEC)	1
arXiv Preprint	1
Archives of Control Sciences	1
ACM/SPEC International Conference on Performance Engineering	1
ACM Transactions on Software Engineering and Methodology	1
ACM Symposium on Applied Computing	1
ACM / IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM)	1
Total	36

for domain modelling and software generation. In the case of tools, DDD has been employed to bridge the gap between business domain concerns and infrastructure, ensuring the maintainability and readability of code.

4.2.5 Distributed Systems. DDD has demonstrated benefits in the development of distributed systems according to Braun et al. [5]. By controlling concurrent data access and achieving semantic compatibility in domain models, DDD has been instrumental in tackling consistency-related design challenges in distributed data-intensive systems.

4.2.6 Service-Oriented Applications. Landre et. al. [28] use DDD techniques to improve the software architecture of a large enterprise system.

4.2.7 System Agnostic. Certain studies have shown that DDD can provide valuable contributions for the implementation of software which is suitable across various system categories [9, 21, 46, 53]. In these studies, DDD principles and patterns have been applied to implement practitioner frameworks

and methods to address communication gaps between business specialists and software designers, improve software system modularity, and support the creation of context maps.

4.2.8 DDD Tools. DDD has been applied to improve code quality, readability, and maintainability and several tools created for this purpose. Perillo et. al. [40] proposes "domain annotations" to separate business domain concerns from infrastructure concerns, while Kapferer et. al. [20] introduces a modelling language and tool (Context Mapper) to effectively create, evolve, and use DDD Context Maps.

4.3 (RQ3) Which specific software development problems have been addressed through the application of DDD in previous studies?

The SLR conducted in this study sought to identify the specific software development problems that have been addressed through the application of DDD in previous research. The results revealed several challenges and motivations encountered in various software development projects, encompassing diverse domains such as microservices architecture, legacy system modernization, distributed systems, and complex decision support systems. The subsequent information present a comprehensive overview of the identified challenges and motivations for adopting DDD.

4.3.1 Addressing API Design Problems in Distributed Systems. One of the prominent challenges addressed by DDD in software development projects is related to API design in distributed systems, particularly in microservice-based systems. This challenge revolves around providing clear guidance on how to derive APIs and API endpoints from domain model elements to facilitate the development of distributed systems [45].

4.3.2 Investigating Performance in Component-Based Software. In the pursuit of achieving optimized performance and resource metrics in component-based software, DDD has been employed in research to investigate the effect of workload patterns. Specifically, the research aimed to gain insights into how patterns like Command Query Responsibility Segregation (CQRS) and Event Sourcing (ES) perform in different scenarios, thereby providing valuable guidelines for architects in selecting appropriate architectures based on workload characteristics [38].

4.3.3 Reducing Complexity in Currency Exchange Systems. A prevalent software development problem addressed by DDD pertains to reducing complexity and high coupling in currency exchange systems. The case in question involves the integration layer between a new currency exchange system (SPOT) and a legacy back office system (TBS), wherein the integration layer (TBSEXP) had become an unwieldy burden with unpredictable side effects following minor changes. To tackle this complexity and streamline the integration process, researchers sought to devise effective solutions using DDD principles [38].

4.3.4 Microservice Size, Partitioning and Defining API Endpoints. Microservice architecture introduces the challenge of determining the appropriate size of individual microservices, thereby necessitating a conceptual methodology for partitioning microservices based on domain engineering techniques. DDD has been instrumental in addressing this challenge, aiming to achieve optimal microservice design and composition within the context of a microservices architecture [19, 45].

4.3.5 Improving Code Quality with Domain Annotations. In the pursuit of enhancing code quality, readability, and maintainability while ensuring a clear separation between the business domain and infrastructure concerns, DDD has been leveraged to address challenges associated with using annotations in DDD. The proposed concept of "domain annotations" by Perillo et al. [40] serves to mitigate these challenges and fosters a robust software design approach.

4.3.6 Enhancing Software Construction and Development Environment. DDD has been instrumental in addressing critical software construction issues, particularly concerning generative, module-based software construction, and seamless integration within the development environment. Notably, in the work of Le et al. [31] the adoption of the jDomainApp framework and an Eclipse IDE plugin has been proposed as a solution to these challenges, thereby optimizing software construction and development processes.

4.3.7 Multi-Platform SPA Development. The challenges faced by Single Page Application (SPA) developers encompass two crucial aspects: designing SPAs that can effectively work across different SPA frameworks (e.g., Angular, React, React Native, and Vue.js) and translating this design into an intermediate high-level language that can seamlessly transform into a target framework of choice. DDD offers a multi-platform, hierarchical approach to tackle these challenges, significantly enhancing SPA development productivity [32].

4.3.8 Scaling Agility in Large Organizations. Large organizations encounter a myriad of challenges when striving to scale agility, including effective coordination and communication between agile teams, inter-team dependencies, and the need for clearly defined requirements. By adopting Domain-Driven Design, enterprises have sought to address these challenges and provide architectural guidance to support large-scale agile development. However, it is important to acknowledge that successful DDD implementation in large organizations necessitates experienced enterprise architects to provide effective guidance [48].

4.3.9 Complexities in Blockchain-Based Traceability Systems. The design and development of blockchain-based traceability systems (BTSs) pose unique challenges and complexities, including domain model intricacies, modeling boundaries, and managing the relationship between different bounded contexts. Researchers have employed DDD and microservice architecture to address these challenges, thereby improving the cohesiveness, maintainability, and extensibility of BTSs in handling complex data in the global supply chain in [55].

4.3.10 Challenges in Model Decomposition. Decomposing software systems into cohesive and loosely coupled modules poses significant challenges for software engineers and service designers. DDD patterns have been instrumental in addressing this challenge, offering a clear and concise interpretation of these patterns, despite debates and differing interpretations among practitioners [21].

4.3.11 Legacy System Modernization and Refactoring. In the context of modernizing legacy systems, DDD has emerged as a valuable approach to address the communication gap between business specialists and software designers. By bridging this gap, DDD enables the alignment of requirements and conceptual models, thereby contributing to improved system design and development [53, 58].

4.4 (RQ4) What are the most common DDD techniques utilized in previous research, such as Anti-Corruption Layers, Bounded Contexts, and other related approaches?

In this SLR on DDD, we investigated the most commonly utilized DDD techniques across various software systems. The analysis was based on a dataset comprising studies identified through our study selection process explained in Section 2.4. Each study was examined for the presence or absence of specific DDD techniques, marked by "Y" for used and "N" for not used techniques during our data extraction process. Our results are also visually shown in Figure 5.

The results reveal that certain DDD techniques are prevalent in the majority of the analyzed studies, indicating their significance and widespread adoption in the DDD context. The following DDD techniques were found to be most commonly utilized:

1. *Ubiquitous Language*. Ubiquitous language is a fundamental DDD technique, and our findings indicate that it is frequently employed in various software systems, including Microservices, Enterprise Software Systems and Web Applications [10, 18, 19, 21, 25, 32, 34, 34, 37–39, 48, 58].
2. *Bounded Context*. Bounded contexts were found to be widely utilized in Microservices, Enterprise Software Systems and Web Applications [4, 5, 7, 19, 21, 32, 34, 38, 39, 46, 48, 57, 58].
3. *Entities and Value Objects*. Entities and value objects, representing domain concepts, were extensively adopted in Microservices, Enterprise Software Systems and Web Applications [5, 9, 10, 14, 16, 18, 19, 21, 32, 34, 38, 39, 46, 48, 53, 55, 58].

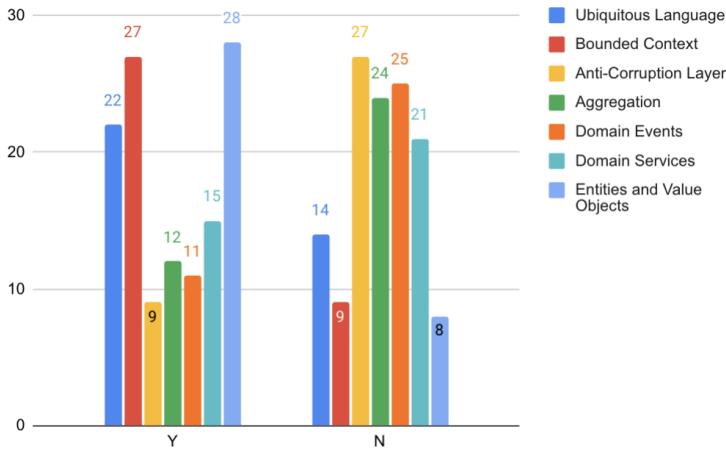


Fig. 5. Breakdown of the applied DDD techniques in the primary studies. Applied DDD techniques marked with "Y" and not applied techniques marked with "N".

In contrast, the *Anti-Corruption Layer* was identified as the least used DDD technique across different software systems. Several studies did not utilize the anti-corruption layer in their respective projects [6, 17, 20, 24, 28, 31, 38]. The *Anti-Corruption Layer* serves as a means to protect the integrity of the main domain from external and legacy systems, and its limited adoption suggests that certain software projects may have addressed integration challenges through alternative approaches or may not have encountered a specific need for this technique.

4.5 (RQ5) What are the challenges encountered during the implementation of DDD in software development projects?

The challenges encountered during the implementation of DDD in software development projects are diverse and demand careful consideration to ensure successful adoption. In this RQ, we present the key challenges identified through primary studies.

4.5.1 Addressing Design Complexities. Several studies highlight the complexity associated with implementing DDD patterns, particularly in the context of microservices and distributed systems. The integration layer between different Bounded Contexts can become burdensome, leading to high coupling and unpredictability in the system. This challenge, for instance, is exemplified in the case of the currency exchange system (SPOT) and the legacy back-office system (TBS) in Peng et al. [38]. Finding the right balance between high cohesion within services and loose coupling between them is crucial for scalability and maintainability [19, 38].

4.5.2 Identifying Optimal Microservices Boundaries. Decomposing monolithic systems into microservices presents its own set of challenges. Determining the appropriate size and boundaries of microservices is not straightforward and requires careful consideration of the domain and workload characteristics [19]. The granularity of microservices can be subjective, leading to potential disagreements among development teams [21]. Ensuring that microservices align with business needs and sub-domains can be challenging [16]. Additionally, finding suitable examples that demonstrate DDD-driven microservices architecture is often noted as a limitation [52].

4.5.3 Managing Domain Model Complexity. When adopting DDD, managing the complexity of the domain model becomes a crucial challenge. Modeling boundaries between different Bounded Contexts and ensuring the relationships between them are properly managed require deep knowledge about the organization's domain [9]. The complexity of the domain model and the interactions between bounded contexts can hinder the process of abstraction and generalization of elements [9]. This challenge is especially pronounced when dealing with large-scale and complex data-intensive systems [55].

4.5.4 Integrating DDD with Existing Frameworks. Integrating DDD into existing software frameworks can present compatibility issues, as DDD introduces encapsulation and abstraction levels that may not align seamlessly with existing structures [58]. Refactoring in an actively developed codebase can also prove challenging, especially when engineers lack prior experience with DDD [58].

4.5.5 Communication Gap between Business Specialists and Software Designers. Incorporating DDD requires effective communication between business specialists and software designers to ensure consistency in requirements and conceptual models [53]. Misalignment in communication can lead to discrepancies in the final design and implementation.

4.5.6 Model-Code Gap in Domain-Specific Language (DSL). Studies that focus on DSL-based DDD implementations point out potential weaknesses, including the existence of a "model-code" gap [20]. The lack of a deeper knowledge about the domain may hinder the process of abstraction and generalization, leading to artifacts that may not align with the organization's domain [9].

4.5.7 Impact of Developer's Experience and Challenges in Onboarding. The level of experience and familiarity with DDD concepts among the development team can influence the effectiveness of DDD implementation [16]. Developers with prior experience in DDD are more likely to understand and apply DDD patterns effectively, resulting in better-designed and more maintainable software systems [16]. On the other hand, inexperienced developers may struggle with grasping the complexities of DDD and may not fully utilize the benefits of the approach [16]. Also introducing DDD to a development team that has little or no prior exposure to DDD can be a challenge [58]. Onboarding developers and providing the necessary training and resources to understand DDD concepts and principles can require additional time and effort [58]. This challenge can be particularly pronounced in organizations that are transitioning from traditional software development approaches to DDD.

4.5.8 Navigating Controversial Debates. Controversial debates and differing interpretations exist among practitioners regarding the application and combination of DDD patterns [21, 58]. Developers may face challenges in deciding which patterns to adopt and how to effectively integrate them into the development process. This can result in disagreements within the team and may require additional effort to reach a consensus.

4.5.9 Need for Experienced Domain Experts. Incorporating DDD requires a deep understanding of the domain and business requirements. Experienced domain experts are valuable in guiding the

development team to create accurate and effective domain models [52]. However, the availability of such domain experts may be limited, especially in specialized domains, and this can hinder the DDD adoption process.

4.6 (RQ6) How has the effectiveness of DDD been assessed and measured in prior studies?

The effectiveness of Domain-Driven Design (DDD) has been assessed and measured in prior studies through various evaluation methods and metrics. It is evident from the data that prior studies have employed a combination of quantitative and qualitative evaluation methods to assess the effectiveness of DDD. These methods are described in more detail in the following subsections.

4.6.1 Performance and Resource Metrics. In the case of Maddodi et al. [34] the effectiveness of DDD was measured by evaluating the performance and resource metrics of the software system using Command Query Responsibility Segregation (CQRS) and Event Sourcing (ES) frameworks. The study reported relative error values for throughput and response time for different scenarios.

4.6.2 Feedback from Stakeholders. In [48] the effectiveness of DDD practices, such as event storming workshops and domain modelling, was measured through the adoption and use of these practices by agile teams. The study also considered feedback from stakeholders, including Program Managers, Product Owners, and Domain Architects, on the benefits and value of DDD in supporting large-scale agile software development.

4.6.3 Context Mapping and Complexity Reduction. Landre et al. [28] evaluated the effectiveness of DDD, particularly the strategic part of it, by using context mapping and context relationships to analyze the software architecture and identify complexities. The study demonstrated how DDD provides mechanisms to address the identified problems and improve the quality of the Enterprise Architecture and derived software architectures by applying responsibility layers and improving scoping of projects.

4.6.4 Effectiveness of Design Guidelines. Braun et al. [5] conducted an assessment to determine the efficacy of design guidelines pertaining to distributed data-intensive systems. The evaluation encompassed various metrics, including the proportion of compatible domain operations and the extent of trivial aggregates within the domain model.

4.6.5 Questionnaires and Developer Perception. In a seminal work by Ozkan et al. [58], the efficacy of DDD was evaluated using a combination of qualitative methods, including focus group sessions, semi-structured interviews, and think-aloud protocols. Additionally, a quantitative analysis was conducted using the Technology Acceptance Model (mTAM) questionnaire. The findings revealed that DDD positively impacted software maintainability as perceived by developers. Similarly, Braun et al. [5] undertook an assessment of design guidelines effectiveness by gauging developers' perceptions in their study.

4.6.6 Measuring Effectiveness of ECD3 Guidelines. In Braun et al. [4] the effectiveness of the DDD measured based on Eventually Consistent Domain-Driven Design Guidelines (ECD3), assessed by conducting a comparative analysis between compatible and incompatible domain operations in the redesigned domain models. The implementation of the recommended ECD3 guidelines exhibited a notable enhancement in the proportion of compatible operations within the domain models.

4.6.7 Software Generation and Complexity Analysis. The effectiveness of the DDD-based software generation method proposed in [31] was evaluated by examining its linear time complexity

and scalability on real-world software of considerable magnitude, through the utilization of the jDomainApp framework.

4.6.8 Time and Resource Consumption. In [17] the effectiveness of including business domain concepts in the Domain-Specific Language (DSL) was measured by evaluating the perceived quality, creation time, and length of test case specifications.

4.6.9 Granularity and Coupling Analysis. The study [52] assessed the effectiveness of DDD in microservices by comparing the coupling and cohesion values of different examples. The study concluded that original examples achieved a good balance between low coupling and acceptable cohesion.

4.6.10 Prototype Application. In [25] the effectiveness of the approach for modernizing legacy lottery applications into microservices was evaluated through its successful application to a real-world lottery application.

4.6.11 Metamodel and Design Models. Le et al. [32] aimed to assess the efficacy of DDD in constructing Single Page Applications (SPAs) compatible with multiple platforms. For this purpose, they developed a novel metamodel and implemented a hierarchical approach within the DDD framework. The outcomes of the study revealed that employing the domain model as a central component and adhering to the hierarchical DDD methodology yielded favorable design models alongside prosperous generation of SPAs tailored for various platforms.

4.7 (RQ7) Who are the key stakeholders involved and benefited in the implementation of DDD?

In the implementation of DDD, several key stakeholders are involved to ensure its successful application. These stakeholders encompass a diverse set of roles, including software engineers, architects, project managers, and domain experts. Through our SLR, we have identified the following key stakeholders.

4.7.1 Software Engineers (SWEs). SWEs play a central role in implementing DDD principles and practices in various software systems. They are responsible for translating the domain model into code, implementing domain-specific logic, and ensuring the alignment of the software architecture with the domain requirements [34, 38, 45, 46, 58].

4.7.2 Architects. Architects are crucial in the implementation of DDD as they provide strategic guidance and oversight on the overall software design. They collaborate with SWEs and other stakeholders to define the high-level architecture, bounded contexts, and domain models that capture the domain knowledge effectively [9, 21, 34].

4.7.3 Project Managers. Project managers play a vital role in coordinating DDD implementation efforts. They ensure that DDD practices align with project goals, allocate resources appropriately, and facilitate effective communication between different teams and stakeholders [38, 48].

4.7.4 Domain Experts. Domain experts are individuals who possess in-depth knowledge of the specific business domain for which the software system is being developed. Their active involvement is critical for refining the domain model, identifying key domain concepts, and validating the software's alignment with real-world domain requirements [9, 45, 48].

In addition to the primary stakeholders mentioned above, the literature also identifies other relevant participants involved in the implementation of DDD. These participants include Quality Assurance (QA) Engineers, DevOps Engineers, UX Designers, Business Analysts, Systems Analysts, Data Modeling Specialists, and end-users [4, 6, 17, 37, 38, 58].

Taking our findings in consideration, we identified that the successful implementation of DDD requires close collaboration and communication among these diverse stakeholders to ensure that the developed software system effectively represents the domain’s complexity and meets the business needs. The active engagement of software engineers, architects, project managers, and domain experts, along with the contributions from other relevant participants, enables the application of DDD principles to create robust and domain-centric software solutions.

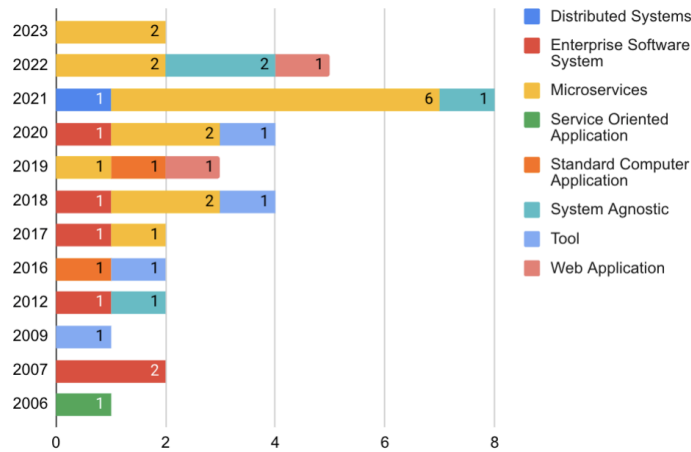


Fig. 6. DDD studies grouped by their software systems and publication years.

5 DISCUSSIONS

In this section, we aim to consolidate and evaluate the extensive range of information obtained through our RQs in order to present a comprehensive overview of the effectiveness, challenges, and potential consequences associated with the adoption of DDD in software development. Throughout this section, we will do an examination of the findings by establishing connections across various studies and identifying recurring themes and patterns. It is important to acknowledge that certain aspects of the discussion have already been addressed in Section 4 when answering to the RQs. Nevertheless, within this dedicated section, we will explore further nuances, ramifications, and potential applications attributable to DDD based on the available evidence.

5.1 Adoption and Implementation of DDD

Several studies report successful implementations of DDD, showcasing its ability to improve software design, maintainability, scalability, and agility. DDD concepts, such as Ubiquitous Language, Bounded Contexts, Aggregates, Entities, Value Objects, and Domain Events, are commonly utilized to model complex domains effectively. Many researchers emphasize the significance of defining clear domain boundaries and using a common language between domain experts and software designers to ensure effective communication and understanding. Additionally, DDD is frequently integrated with other software development paradigms, such as microservices and event-driven architecture, to achieve better modularity and maintainable systems.

5.2 Ubiquitous Language as a Core Principle of DDD

The data analyzed points out that the Ubiquitous Language stands as a foundational principle of DDD. It ensures a shared and consistent understanding of the domain between stakeholders,

including business specialists, software designers, and developers [13]. The studies consistently emphasize the importance of Ubiquitous Language in facilitating communication, clarifying domain concepts, and improving collaboration among team members [10, 12, 16, 19, 30–32, 34, 39, 40, 45, 48, 53, 58].

5.3 Bounded Context and Decomposition of Microservices

Bounded Context is another fundamental concept in DDD, and its significance in microservices architecture is evident in several studies [5, 6, 18, 19, 25, 34, 38, 39, 48, 52]. The concept of Bounded Context is used to define explicit boundaries where specific domain models apply, allowing the system to be decomposed into manageable, cohesive units. Decomposing a system into microservices based on Bounded Contexts enables easier scaling, maintenance, and independent updating of each microservice.

5.4 Domain Services, Entities, and Value Objects as a Tactical DDD

Several studies discuss the use of Domain Services, Entities, and Value Objects as part of DDD's tactical patterns [16, 32, 34, 39, 48, 58]. Domain Services are employed to handle application tasks and technical code that is not part of the core business logic, while Entities and Value Objects represent different entities and their relationships in the domain model. These concepts contribute to modeling complex business domains and enabling object-oriented representations of domain concepts.

5.5 Anti-Corruption Layer for Maintaining Domain Integrity

When dealing with different bounded contexts, interacting with external systems or refactoring legacy systems, the risk of corrupting a domain model with concepts and structures from another context is high. The Anti-Corruption Layer acts as a safeguard, ensuring that each context's domain remains untainted and independent. The concept of the Anti-Corruption Layer is discussed in some studies [19, 28, 30, 38]. It is utilized in the studies to manage dependencies between different domains, preventing the contamination of contexts, safeguard the domain model of a Bounded Context from changes in other contexts, segregating application-specific elements from domain-specific components in microservices and maintaining consistency between aggregates. Studies also suggest that Anti-Corruption Layer shields new contexts from potential contaminations or disruptions originating in legacy contexts.

5.6 All Hands on Deck: It is not Only About Software Engineers

Based on the data extracted from the literature, the implementation of Domain-Driven Design (DDD) involves the active participation of various key stakeholders who play critical roles in ensuring its successful application. These stakeholders encompass a diverse set of roles, including Software Engineers (SWEs), Architects, Project Managers, and Domain Experts. SWEs hold a central position in the DDD implementation process, responsible for translating the domain model into executable code, implementing domain-specific logic, and aligning the software architecture with domain requirements [34, 38, 45, 46, 58]. Architects contribute strategic guidance and oversight to the overall software design, collaborating with SWEs and other stakeholders to define high-level architecture, bounded contexts, and domain models that effectively capture domain knowledge [9, 21, 34].

Project Managers play a vital role in coordinating DDD implementation efforts, ensuring alignment with project goals, resource allocation, and effective communication between teams and stakeholders [38, 48]. Domain Experts, possessing in-depth knowledge of the specific business

domain, actively contribute to refining the domain model, identifying key domain concepts, and validating the software's alignment with real-world domain requirements [9, 45, 48].

Moreover, the literature also identifies other relevant participants involved in the DDD implementation, including QA Engineers, DevOps Engineers, UX Designers, Business Analysts, Systems Analysts, Data Modeling Specialists, and end-users [4, 6, 17, 37, 38, 58]. The successful implementation of DDD requires close collaboration and communication among these diverse stakeholders to ensure that the developed software system effectively represents the domain's complexity and meets the business needs. The active engagement of software engineers, architects, project managers, and domain experts, along with the contributions from other relevant participants, enables the application of DDD principles to create robust and domain-centric software solutions. The involvement of these stakeholders enhances the alignment of software systems with real-world domain requirements and fosters the development of high-quality and effective software solutions.

5.7 Evaluation of DDD Effectiveness

The evaluation of DDD effectiveness varies across studies. Some studies present empirical evaluations based on metrics, user feedback, or controlled experiments [5, 10, 34, 35, 45, 48, 58]. However, several studies lack specific evaluation results or empirical evidence [1, 6, 12, 25, 40, 53].

To establish a more comprehensive understanding of the benefits and potential challenges of implementing DDD in different software development scenarios, further empirical research is needed. Researchers and practitioners should focus on collecting concrete metrics, conducting real-world case studies, and measuring the perception of practitioners. We believe that it is also essential to involve human metrics, such as user feedback and assessments from domain experts, architects, and experienced developers. Considering the challenges in implementing DDD, human-centric evaluations can provide valuable insights into the effectiveness of DDD and its impact on the software development process.

5.8 Rise of the Microservices in Domain-Driven Design (DDD)

Our SLR reveals a significant shift in the popularity of microservices in DDD studies starting 2017 and at peak in 2021. As depicted in Figure 6, prior to 2017, credible DDD studies specifically focusing on microservices were scarce, indicating a lack of attention in this area. However, from 2017 onwards, we observed a notable increase in the number of studies exploring the application of DDD principles in the context of microservices. This trend aligns with the findings of a Systematic Mapping Study on Microservices conducted by Hamzehloui et al. [15] in 2018, which also reports a rising number of evaluation papers, suggesting that microservices have been gaining wider implementation in real-world software development scenarios.

Furthermore, Vigiato et al. [51] in 2018 highlights the increasing popularity of microservices architectures, further emphasizing the growing relevance and adoption of microservices in the software industry. In a similar vein, Knoche et al. [23] in 2019 notes that many companies are actively considering whether microservices are a viable option for their applications, indicating the considerable interest and attention surrounding the adoption of microservices-based architectures.

The convergence of our findings with those of the previously mentioned studies and experts further corroborates the growing significance of microservices in the domain of software development, especially in the context of DDD methodologies. The upward trend in microservices adoption is evident from the year 2017 onwards, indicating a turning point in the landscape of software development practices.

5.9 Challenges and Considerations in Implementing DDD Effectively

To ensure the successful implementation of DDD, it is essential to employ DDD techniques accurately and efficiently. Proper utilization of DDD principles demands a skilled team consisting of experienced developers, domain experts, and seasoned architects [58]. The significance of experience in DDD cannot be overstated, as we have observed from various studies that onboarding to DDD is not a straightforward process, and the expertise of developers significantly influences the success of the implementation [21, 58].

Implementing DDD effectively requires a substantial amount of attention and effort, particularly when integrating it into existing software systems. The transition from traditional software development approaches to DDD can be challenging, and the process demands careful planning, training, and a clear understanding of the DDD concepts and principles. Seasoned developers with prior experience in DDD are more likely to grasp the complexities and intricacies of the approach, resulting in better-designed and more maintainable software systems [16]. On the other hand, inexperienced developers may struggle to fully utilize the benefits of DDD, leading to potential issues in the implementation process.

Furthermore, the involvement of domain experts plays a crucial role in the success of DDD implementation. Their deep understanding of the business domain is invaluable in guiding the development team to create accurate and effective domain models [52]. However, acquiring experienced domain experts can be challenging, especially in specialized domains. Their scarcity may pose hurdles in the adoption of DDD, but their involvement is crucial for crafting domain models that truly reflect the intricacies and requirements of the real-world domain.

5.10 Academia-Industry Collaboration

The academic studies in our SLR predominantly emphasize the theoretical aspects of DDD. They explore the foundational principles and patterns of DDD, such as Ubiquitous Language, Bounded Context, Entities, and Value Objects. These studies often propose novel frameworks, tools and methodologies that extend the DDD concepts to specific problem domains. While they provide valuable insights into the theoretical underpinnings of DDD, they might lack empirical validation and practical implementation in real-world projects.

On the other hand, we experienced that the industry-focused studies put DDD principles into practice in various software systems. They showcase how DDD is applied to real-world projects and explore its impact on system design, architecture, and development. These studies often involve case studies and experience reports from software engineers, architects, and domain experts. The emphasis is on practical implementation, challenges faced, and lessons learned while applying DDD in complex software projects.

Our findings indicate a healthy mix of academic and industry studies, reflecting the growing interest in DDD from both research and practical application perspectives. While academic studies contribute theoretical advancements and innovative approaches, industry-focused studies demonstrate the real-world applicability and effectiveness of DDD principles in solving complex software design and development challenges. More interaction between academia and industry in DDD research could potentially facilitate the transfer of knowledge and best practices, contributing to the further advancement and adoption of DDD principles in real-world software development scenarios, as successfully demonstrated in [58].

5.11 A Critical Analysis of Research Quality in the DDD Literature

The analyzed studies demonstrate a varying degree of adherence to DDD principles and the effective application of DDD concepts. Some studies demonstrating effective application of DDD principles,

while others exhibit shortcomings in implementation and evaluation. To enhance research quality, future studies should emphasize the consistent use of Ubiquitous Language, explicit definition of Bounded Contexts, proper utilization of Domain Events and DDD patterns, and robust evaluation methodologies. Additionally, studies should openly discuss the challenges and limitations of DDD, contributing to a more comprehensive understanding of its potential and practical implications in software development.

Several studies were found to have employed the notion of Ubiquitous Language, which facilitates effective communication between stakeholders and software designers by creating a shared vocabulary [13, 50]. However, a few studies failed to implement this language, potentially hindering communication and understanding among team members. To improve the research quality, future studies should emphasize the importance of adopting and consistently applying Ubiquitous Language throughout the development process.

Another key concept in DDD is Bounded Context, which helps define explicit boundaries where specific domain models apply [13, 50]. A number of studies effectively utilized Bounded Context to organize and segregate domains, but some failed to explicitly mention or implement this principle. To enhance the research quality, studies should ensure clear identification and definition of Bounded Contexts, as this is crucial for managing interactions between different models and contexts.

Furthermore, the application of Domain Events to trigger the execution of functionalities in downstream aggregates was observed in some studies. However, not all studies effectively leveraged this concept. Implementing Domain Events can improve event-driven communication between domain components, promoting a more cohesive and flexible architecture [13, 50]. To enhance the quality of research, studies should give due attention to Domain Events and explore their potential benefits in event-driven systems.

Several studies showcased the use of Aggregates, Entities, and Value Objects to model complex business domains. While these concepts were well-implemented in some cases, others lacked specific details or provided inadequate explanations about their usage. To bolster research quality, studies should strive for comprehensive and consistent implementation of these DDD patterns, ensuring a clear understanding of their roles and relationships within the domain model [13, 50].

One significant aspect that emerged from the data was the lack of specific evaluation methods in some studies. While some research incorporated empirical evaluations, others relied solely on proposing methodologies or demonstrating case studies without conducting rigorous evaluations. To enhance research quality, future studies should adopt more robust evaluation methods, such as controlled experiments, surveys, or user feedback, to assess the effectiveness of DDD implementation.

Moreover, a few studies did not discuss the challenges and limitations associated with DDD adoption, leaving potential gaps in addressing concerns or drawbacks of the approach. To improve research quality, studies should acknowledge and discuss both the advantages and limitations of DDD, offering a balanced view to readers and practitioners.

6 LIMITATIONS AND THREATS TO VALIDITY

Some limitations and validity considerations are applicable for SLR studies [41, 42]. The limitations and threats to the validity of this study are discussed in this section.

Timeframe and Currency. One limitation is the timeframe of the literature search and the currency of the included studies. The SLR is based on data available up to June 2023, and relevant studies published after that date might have been missed. The field of DDD is rapidly evolving, and newer developments or trends may not be fully captured in this review.

Quality of Studies. The quality of individual studies can impact the overall reliability of the SLR findings. The included studies vary in terms of methodological rigor, sample size, and

research design. Some studies might have limitations or biases that could influence their results and subsequent conclusions. Nonetheless, we endeavored to rigorously exclude studies of low quality by systematically applying inclusion and exclusion criteria as well as conducting thorough quality assessments.

Inclusion and Exclusion Criteria. The effectiveness of an SLR largely depends on the clarity and appropriateness of the inclusion and exclusion criteria. Despite our efforts to define comprehensive criteria, there is a possibility of unintentionally excluding relevant studies or including studies that do not precisely fit the research questions. To mitigate this potential limitation, a rigorous approach was adopted. Two authors independently applied predefined inclusion and exclusion criteria to a randomly selected pool of 60 studies. The level of agreement between the two authors was assessed using Cohen's Kappa coefficient [8], a widely accepted measure of inter-rater reliability. Encouragingly, the analysis revealed a perfect agreement between the authors in their decisions regarding study inclusion and exclusion within the randomly sampled studies. This robust agreement underscores the reliability of the selection process and enhances the credibility of the findings.

Bias in Data Extraction. A limitation related to the validity of data extraction in this study is the potential for human error or bias during the data extraction process. Despite efforts to maintain consistency and accuracy, the interpretation of information from primary studies could be influenced by the subjectivity of the researchers. Different individuals extracting data may interpret the findings differently or unintentionally introduce errors, leading to inconsistencies in the extracted data. To address this limitation, a careful and systematic approach was employed, and multiple researchers were involved in the data extraction process. Inter-rater reliability checks and discussions among the researchers were conducted to ensure agreement and minimize discrepancies. However, it is essential to acknowledge that the possibility of human-related errors cannot be entirely eliminated, and they may have some impact on the final results and conclusions of the systematic literature review.

Publication Bias. One potential threat to validity is publication bias. The studies included in this SLR were obtained from various academic databases and sources, which might not represent the entire body of literature on DDD. Some studies might not have been published or accessible, leading to a skewed representation of the research landscape. Nevertheless, to mitigate this bias, we performed a comprehensive search across multiple academic databases, including Google Scholar and Scopus during snowballing process. This extensive search approach was aimed at encompassing a wide range of academic sources and reducing potential biases in the selection of studies.

Language Bias. The SLR focused primarily on studies published in English, which might introduce a language bias. Relevant research conducted in other languages could have been omitted, potentially impacting the comprehensiveness of the review.

7 CONCLUSION

This systematic literature review (SLR) provides a comprehensive overview of the effectiveness, challenges, and implications of adopting Domain-Driven Design (DDD) in software development. Through the analysis of a diverse set of primary studies, we identified key stakeholders involved in the implementation of DDD, including Software Engineers (SWEs), Architects, Project Managers, and Domain Experts. These stakeholders play critical roles in ensuring the successful application of DDD principles and practices in various software systems.

The findings highlight the significance of Ubiquitous Language as a core principle of DDD, emphasizing its role in facilitating effective communication and collaboration among team members. Bounded Contexts and the decomposition of microservices stand out as fundamental concepts in DDD, enabling the creation of scalable and maintainable software systems. Domain Services,

Entities, and Value Objects serve as tactical DDD patterns, contributing to the modeling of complex business domains.

The implementation of DDD demands considerable attention and effort, particularly in transitioning from traditional software development approaches to DDD. Experienced developers and domain experts play crucial roles in effectively implementing DDD concepts. The involvement of domain experts is especially critical for crafting domain models that accurately represent real-world domain requirements.

The SLR also highlights the rise of microservices in the context of DDD, indicating a significant shift in the popularity of microservices-based architectures since 2017. However, further empirical research is needed to comprehensively understand the benefits and challenges of implementing DDD in different software development scenarios.

The collaboration between academia and industry in DDD research is evident, with academic studies contributing theoretical advancements and innovative approaches, while industry-focused studies showcase practical implementation and real-world applicability. This collaboration can facilitate knowledge transfer and best practices, contributing to the advancement and adoption of DDD principles in software development.

A critical analysis of research quality in the DDD literature reveals varying degrees of adherence to DDD principles and the effective application of DDD concepts. Future studies should emphasize the consistent use of Ubiquitous Language, clear definition of Bounded Contexts, proper utilization of Domain Events and DDD patterns, and robust evaluation methodologies. Moreover, studies should openly discuss the challenges and limitations of DDD to provide a comprehensive understanding of its potential and practical implications.

Domain-Driven Design offers valuable insights and practices for software development, fostering communication, and collaboration among stakeholders while modeling complex business domains. However, its successful implementation depends on the expertise of stakeholders, proper utilization of DDD principles, and the adoption of robust evaluation methods. As software systems continue to grow in complexity, DDD remains a relevant and promising approach for developing domain-centric and scalable software solutions.

REFERENCES

- [1] Sebastian Abeck, Michael Schneider, Jan-Philip Quirnbach, Heiko Klarl, Christof Urbaczek, and Shkodran Zogaj. 2019. A Context Map as the Basis for a Microservice Architecture for the Connected Car Domain. (2019). https://doi.org/10.18420/INF2019_18 ISBN: 9783885796886 Publisher: Gesellschaft für Informatik e.V..
- [2] Abel Avram and Floyd Marinescu. 2006. *Domain-Driven Design Quickly: A Summary of Eric Evans' Domain-Driven Design*. C4Media.
- [3] A. Bertoni. 2020. DATA-DRIVEN DESIGN IN CONCEPT DEVELOPMENT: SYSTEMATIC REVIEW AND MISSED OPPORTUNITIES. *Proceedings of the Design Society: DESIGN Conference 1* (May 2020), 101–110. <https://doi.org/10.1017/dsd.2020.4>
- [4] Susanne Braun, Annette Bieniusa, and Frank Elberzhager. 2021. Advanced Domain-Driven Design for Consistency in Distributed Data-Intensive Systems. In *Proceedings of the 8th Workshop on Principles and Practice of Consistency for Distributed Data*. ACM, Online United Kingdom, 1–12. <https://doi.org/10.1145/3447865.3457969>
- [5] Susanne Braun, Stefan Deßloch, Eberhard Wolff, Frank Elberzhager, and Andreas Jedlitschka. 2021. Tackling Consistency-related Design Challenges of Distributed Data-Intensive Systems: An Action Research Study. In *Proceedings of the 15th ACM / IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM)*. ACM, Bari Italy, 1–11. <https://doi.org/10.1145/3475716.3475771>
- [6] Hendrik Bunder and Herbert Kuchen. 2019. A model-driven approach for behavior-driven GUI testing. In *Proceedings of the 34th ACM/SIGAPP Symposium on Applied Computing*. ACM, Limassol Cyprus, 1742–1751. <https://doi.org/10.1145/3297280.3297450>
- [7] Matteo Camilli, Carmine Colarusso, Barbara Russo, and Eugenio Zimeo. 2023. Actor-driven Decomposition of Microservices through Multi-level Scalability Assessment. *ACM Transactions on Software Engineering and Methodology* (Feb. 2023), 3583563. <https://doi.org/10.1145/3583563>

- [8] Jacob Cohen. 1960. A Coefficient of Agreement for Nominal Scales. *Educational and Psychological Measurement* 20, 1 (April 1960), 37–46. <https://doi.org/10.1177/001316446002000104>
- [9] Carlos Eduardo Da Silva, Eduardo Luiz Gomes, and Soumya Sankar Basu. 2022. BPM2DDD: A Systematic Process for Identifying Domains from Business Processes Models. *Software* 1, 4 (Sept. 2022), 417–449. <https://doi.org/10.3390/software1040018>
- [10] Paulius Danenas and Gintautas Garsva. 2012. Domain Driven Development and Feature Driven Development for Development of Decision Support Systems. In *Information and Software Technologies*, Tomas Skersys, Rimantas Butleris, and Rita Butkiene (Eds.). Vol. 319. Springer Berlin Heidelberg, Berlin, Heidelberg, 187–198. https://doi.org/10.1007/978-3-642-33308-8_16 Series Title: Communications in Computer and Information Science.
- [11] Heleen De Goey, Per Hilletoft, and Lars Eriksson. 2019. Design-driven innovation: a systematic literature review. *European Business Review* 31, 1 (Jan. 2019), 92–114. <https://doi.org/10.1108/EBR-09-2017-0160>
- [12] Yu Ding, LiLing Wang, Shaokun Li, XuTong Wang, and JianWei Zhang. 2020. Enterprise service application architecture based on Domain Driven Model Design. In *2020 2nd International Conference on Information Technology and Computer Application (ITCA)*. IEEE, Guangzhou, China, 778–784. <https://doi.org/10.1109/ITCA52113.2020.00167>
- [13] Eric Evans. 2004. *Domain-driven design: tackling complexity in the heart of software*. Addison-Wesley, Boston.
- [14] Pär Hammarström and Erik Herzog. 2016. Experience from integrating Domain Driven Software System Design into a Systems Engineering Organization. *INCOSE International Symposium* 26, 1 (July 2016), 1192–1203. <https://doi.org/10.1002/j.2334-5837.2016.00220.x>
- [15] Mohammad Sadeh Hamzehloui, Shamsul Sahibuddin, and Khalil Salah. 2018. A Systematic Mapping Study on Microservices. In *Advances in Intelligent Systems and Computing*. Springer International Publishing, 1079–1090. https://doi.org/10.1007/978-3-319-99007-1_100
- [16] Benjamin Hippchen, Pascal Giessler, Roland Heinz Steinegger, Michael Schneider, and Sebastian Abeck. 2017. Designing Microservice-Based Applications by Using a Domain-Driven Design Approach. (2017).
- [17] Florian Häser, Michael Felderer, and Ruth Breu. 2016. Is business domain language support beneficial for creating test case specifications: A controlled experiment. *Information and Software Technology* 79 (Nov. 2016), 52–62. <https://doi.org/10.1016/j.infsof.2016.07.001>
- [18] Munezero Immaculee Joselyne, Gaurav Bajpai, and Frederic Nzanywayingoma. 2021. A Systematic Framework of Application Modernization to Microservice based Architecture. In *2021 International Conference on Engineering and Emerging Technologies (ICEET)*. IEEE, Istanbul, Turkey, 1–6. <https://doi.org/10.1109/ICEET53442.2021.9659783>
- [19] Munezero Immaculé Josélyne, Doreen Tuheirwe-Mukasa, Benjamin Kanagwa, and Joseph Balikuddembe. 2018. Partitioning microservices: a domain engineering approach. In *Proceedings of the 2018 International Conference on Software Engineering in Africa*. ACM, Gothenburg Sweden, 43–49. <https://doi.org/10.1145/3195528.3195535>
- [20] Stefan Kapferer and Olaf Zimmermann. 2020. Domain-driven service design. In *Service-Oriented Computing*. Springer International Publishing, Cham, 189–208.
- [21] Stefan Kapferer and Olaf Zimmermann. 2021. Domain-Driven Architecture Modeling and Rapid Prototyping with Context Mapper. In *Model-Driven Engineering and Software Development*, Slimane Hammoudi, Luis Ferreira Pires, and Bran Selić (Eds.). Vol. 1361. Springer International Publishing, Cham, 250–272. https://doi.org/10.1007/978-3-030-67445-8_11 Series Title: Communications in Computer and Information Science.
- [22] Barbara Kitchenham and Stuart Charters. 2007. Guidelines for performing Systematic Literature Reviews in Software Engineering. 2 (Jan. 2007).
- [23] Holger Knoche and Wilhelm Hasselbring. 2019. Drivers and Barriers for Microservice Adoption – A Survey among Professionals in Germany. *Enterprise Modelling and Information Systems Architectures (EMISAJ)* (Jan. 2019), 1:1–35 Pages. <https://doi.org/10.18417/EMISA.14.1> Artwork Size: 1:1–35 Pages Publisher: Enterprise Modelling and Information Systems Architectures (EMISAJ).
- [24] Maciej Koryl. 2017. Active resources concept of computation for enterprise software. *Archives of Control Sciences* 27, 2 (June 2017), 279–291. <https://doi.org/10.1515/acsc-2017-0018>
- [25] Alexander Krause, Christian Zirkelbach, Wilhelm Hasselbring, Stephan Lenga, and Dan Kroger. 2020. Microservice Decomposition via Static and Dynamic Analysis of the Monolith. In *2020 IEEE International Conference on Software Architecture Companion (ICSA-C)*. IEEE, Salvador, Brazil, 9–16. <https://doi.org/10.1109/ICSA-C50368.2020.00011>
- [26] Marco Kuhrmann, Daniel Méndez Fernández, and Maya Daneva. 2017. On the pragmatic design of literature studies in software engineering: an experience-based guideline. *Empirical Software Engineering* 22, 6 (Dec. 2017), 2852–2891. <https://doi.org/10.1007/s10664-016-9492-y>
- [27] Einar Landre, Harald Wesenberg, and Jorn Olmheim. 2007. Agile enterprise software development using domain-driven design and test first. In *Companion to the 22nd ACM SIGPLAN conference on Object-oriented programming systems and applications companion*. ACM, Montreal Quebec Canada, 983–993. <https://doi.org/10.1145/1297846.1297967>
- [28] Einar Landre, Harald Wesenberg, and Harald Ronneberg. 2006. Architectural improvement by use of strategic level domain-driven design. In *Companion to the 21st ACM SIGPLAN symposium on Object-oriented programming systems*,

- languages, and applications*. ACM, Portland Oregon USA, 809–814. <https://doi.org/10.1145/1176617.1176728>
- [29] Duc Minh Le, Duc-Hanh Dang, and Viet-Ha Nguyen. 2016. Domain-driven design patterns: A metadata-based approach. In *2016 IEEE RIVF International Conference on Computing & Communication Technologies, Research, Innovation, and Vision for the Future (RIVF)*. IEEE, Hanoi, 247–252. <https://doi.org/10.1109/RIVF.2016.7800302>
- [30] Duc Minh Le, Duc-Hanh Dang, and Viet-Ha Nguyen. 2018. On domain driven design using annotation-based domain specific language. *Computer Languages, Systems & Structures* 54 (Dec. 2018), 199–235. <https://doi.org/10.1016/j.cl.2018.05.001>
- [31] Duc Minh Le, Duc-Hanh Dang, and Ha Thanh Vu. 2019. jDomainApp: A Module-Based Domain-Driven Software Framework. In *Proceedings of the Tenth International Symposium on Information and Communication Technology - SoICT 2019*. ACM Press, Hanoi, Ha Long Bay, Viet Nam, 399–406. <https://doi.org/10.1145/3368926.3369657>
- [32] Duc Minh Le, Anh Phuong Nguyen, Linh Quang Tran, Hong Thi Le, and Hai Van-Anh Tong. 2022. Generating Multi-platform Single Page Applications: A Hierarchical Domain-Driven Design Approach. In *The 11th International Symposium on Information and Communication Technology*. ACM, Hanoi Vietnam, 344–351. <https://doi.org/10.1145/3568562.3568566>
- [33] Daniel Lübke, Olaf Zimmermann, Cesare Pautasso, Uwe Zdun, and Mirko Stocker. 2019. Interface Evolution Patterns: Balancing Compatibility and Extensibility across Service Life Cycles. In *Proceedings of the 24th European Conference on Pattern Languages of Programs (EuroPlop '19)*. Association for Computing Machinery, New York, NY, USA. <https://doi.org/10.1145/3361149.3361164> event-place: Irsee, Germany.
- [34] Gururaj Maddodi, Slinger Jansen, and Michiel Overeem. 2020. Aggregate Architecture Simulation in Event-Sourcing Applications using Layered Queuing Networks. In *Proceedings of the ACM/SPEC International Conference on Performance Engineering*. ACM, Edmonton AB Canada, 238–245. <https://doi.org/10.1145/3358960.3375797>
- [35] Benjamin Mayer and Rainer Weinreich. 2018. An Approach to Extract the Architecture of Microservice-Based Software Systems. In *2018 IEEE Symposium on Service-Oriented System Engineering (SOSE)*. IEEE, Bamberg, 21–30. <https://doi.org/10.1109/SOSE.2018.00012>
- [36] Jimmy Nilsson. 2006. *Applying domain-driven design and patterns: with examples in C# and .NET*. Addison-Wesley, Upper Saddle River, NJ. OCLC: ocm63703904.
- [37] Peter Oukes, Marc Van Andel, Erwin Folmer, Rohan Bennett, and Christiaan Lemmen. 2021. Domain-Driven Design applied to land administration system development: Lessons from the Netherlands. *Land Use Policy* 104 (May 2021), 105379. <https://doi.org/10.1016/j.landusepol.2021.105379>
- [38] Sam Peng and Ying Hu. 2007. IAnticorruption: a domain-driven design approach to more robust integration. In *Companion to the 22nd ACM SIGPLAN conference on Object-oriented programming systems and applications companion*. ACM, Montreal Quebec Canada, 976–982. <https://doi.org/10.1145/1297846.1297966>
- [39] Pedro Pereira and António Rito Silva. 2022. Towards Transactional Causal Consistent Microservices Business Logic. <http://arxiv.org/abs/2212.11658> arXiv:2212.11658 [cs].
- [40] José Roberto C. Perillo, Eduardo M. Guerra, and Clovis T. Fernandes. 2009. Daileon: a tool for enabling domain annotations. In *Proceedings of the Workshop on AOP and Meta-Data for Software Evolution*. ACM, Genova Italy, 1–4. <https://doi.org/10.1145/1562860.1562867>
- [41] Kai Petersen, Robert Feldt, Shahid Mujtaba, and Michael Mattsson. 2008. Systematic Mapping Studies in Software Engineering. <https://doi.org/10.14236/ewic/EASE2008.8>
- [42] Kai Petersen, Sairam Vakkalanka, and Ludwik Kuzniarz. 2015. Guidelines for conducting systematic mapping studies in software engineering: An update. *Information and Software Technology* 64 (Aug. 2015), 1–18. <https://doi.org/10.1016/j.infsof.2015.03.007>
- [43] Jennie Popay, Helen Roberts, Amanda Sowden, Mark Petticrew, Lisa Arai, Mark Rodgers, Nicky Britten, Katrina Roen, and Steven Duffy. 2006. *Guidance on the conduct of narrative synthesis in systematic reviews: A product from the ESRC Methods Programme*. Lancaster University. <https://doi.org/10.13140/2.1.1018.4643>
- [44] Apitchaka Singjai, Uwe Zdun, and Olaf Zimmermann. 2021. Practitioner Views on the Interrelation of Microservice APIs and Domain-Driven Design: A Grey Literature Study Based on Grounded Theory. In *2021 IEEE 18th International Conference on Software Architecture (ICSA)*. IEEE, Stuttgart, Germany, 25–35. <https://doi.org/10.1109/ICSA51549.2021.00011>
- [45] Apitchaka Singjai, Uwe Zdun, Olaf Zimmermann, and Cesare Pautasso. 2021. Patterns on Deriving APIs and their Endpoints from Domain Models. In *26th European Conference on Pattern Languages of Programs*. ACM, Graz Austria, 1–15. <https://doi.org/10.1145/3489449.3489976>
- [46] Monique Snoeck and Yves Wautelet. 2022. Agile MERODE: a model-driven software engineering method for user-centric and value-based development. *Software and Systems Modeling* 21, 4 (Aug. 2022), 1469–1494. <https://doi.org/10.1007/s10270-022-01015-y>
- [47] Wesley Torres, Mark G. J. Van Den Brand, and Alexander Serebrenik. 2021. A systematic literature review of cross-domain model consistency checking by model management tools. *Software and Systems Modeling* 20, 3 (June 2021),

- 897–916. <https://doi.org/10.1007/s10270-020-00834-1>
- [48] Ömer Uludağ, Matheus Hauser, Martin Kleehaus, Christina Schimpfle, and Florian Matthes. 2018. Supporting Large-Scale Agile Development with Domain-Driven Design. In *Agile Processes in Software Engineering and Extreme Programming*, Juan Garbajosa, Xiaofeng Wang, and Ademar Aguiar (Eds.). Vol. 314. Springer International Publishing, Cham, 232–247. https://doi.org/10.1007/978-3-319-91602-6_16 Series Title: Lecture Notes in Business Information Processing.
- [49] Vaughn Vernon. 2013. *Implementing domain-driven design*. Addison-Wesley, Upper Saddle River, NJ. OCLC: ocn830008330.
- [50] Vaughn Vernon. 2016. *Domain-driven design distilled*. Addison-Wesley, Boston. OCLC: ocn935986578.
- [51] Markos Viggiano, Ricardo Terra, Henrique Rocha, Marco Tulio Valente, and Eduardo Figueiredo. 2018. Microservices in Practice: A Survey Study. (2018). <https://doi.org/10.48550/ARXIV.1808.04836> Publisher: arXiv Version Number: 1.
- [52] Hulya Vural and Murat Koyuncu. 2021. Does Domain-Driven Design Lead to Finding the Optimal Modularity of a Microservice? *IEEE Access* 9 (2021), 32721–32733. <https://doi.org/10.1109/ACCESS.2021.3060895>
- [53] Fernando Wanderley and Denis Silva Da Silveria. 2012. A Framework to Diminish the Gap between the Business Specialist and the Software Designer. In *2012 Eighth International Conference on the Quality of Information and Communications Technology*. IEEE, Lisbon, TBD, Portugal, 199–204. <https://doi.org/10.1109/QUATIC.2012.9>
- [54] Simin Wang, Liguang Huang, Amiao Gao, Jidong Ge, Tengfei Zhang, Haitao Feng, Ishna Satyarth, Ming Li, He Zhang, and Vincent Ng. 2023. Machine/Deep Learning for Software Engineering: A Systematic Literature Review. *IEEE Transactions on Software Engineering* 49, 3 (March 2023), 1188–1231. <https://doi.org/10.1109/TSE.2022.3173346>
- [55] Yanze Wang, Shanshan Li, Huikun Liu, He Zhang, and Bo Pan. 2022. A Reference Architecture for Blockchain-based Traceability Systems Using Domain-Driven Design and Microservices. In *2022 29th Asia-Pacific Software Engineering Conference (APSEC)*. IEEE, Japan, 269–278. <https://doi.org/10.1109/APSEC57359.2022.00039>
- [56] Claes Wohlin. 2014. Guidelines for snowballing in systematic literature studies and a replication in software engineering. In *Proceedings of the 18th International Conference on Evaluation and Assessment in Software Engineering*. ACM, London England United Kingdom, 1–10. <https://doi.org/10.1145/2601248.2601268>
- [57] Junfeng Zhao and Ke Zhao. 2021. Applying Microservice Refactoring to Object-Oriented Legacy System. In *2021 8th International Conference on Dependable Systems and Their Applications (DSA)*. IEEE, Yinchuan, China, 467–473. <https://doi.org/10.1109/DSA52907.2021.00070>
- [58] Ozan Özkan, Önder Babur, and Mark Van Den Brand. 2023. Refactoring with domain-driven design in an industrial context: An action research report. *Empirical Software Engineering* 28, 4 (July 2023), 94. <https://doi.org/10.1007/s10664-023-10310-1>

Received 1 January 0000; revised 1 January 0000; accepted 1 January 0000